

Empirical Analysis for “Predictive conditional alphas” (Distaso et. al 2026)

Paper authors: Valentina Corradi, Walter Distaso,
Marcelo Fernandes, and Asger Lunde
Research assistant: Ricardo Semião e Castro

14 Jul 2026

Table of contents

1	Empirical Analysis for “Predictive conditional alphas” (distaso et. al 2026)	2
1.1	Repository organization	2
2	Data processing: factors	3
2.1	Data and methodology	3
2.2	Code setup	4
2.3	Importing and dates handling	4
2.4	Log-returns and 5-minute aggregation	6
3	Data processing: stocks	8
3.1	Data and methodology	8
3.2	Code setup	9
3.3	Initial data cleaning	10
3.4	Dates analysis	13
3.5	Aggregation of 1-minute log returns	19
3.6	Appendix	22
4	Factor PCA	26
5	Realized betas and risk-adjusted returns	27
6	Conditional alphas	28

1 Empirical Analysis for “Predictive conditional alphas” (distaso et al. 2026)

This project contains the empirical analysis for the paper “Predictive conditional alphas” by distaso et al. (2026). The relevant version of the paper can be found [bibliography/distaso2026_JASA.pdf](#).

The general steps of the empirical analysis are:

- Download and process factor and stock data.
- Extract high-frequency principal components from the factors.
- Estimate realized betas from the stock returns and the principal components.
- Estimate conditional alphas from the risk-adjusted returns.

The detailed methodology for each step is described in the associated report chapter or `.ipynb` notebook in `src/`.

This project has a repository and a PDF report associated. Both are explained in this text. Some links point to files in the repository and won’t work in the PDF report.

The project is run in python 3.14 and DuckDB. The `uv` config files allow for running the code with the correct package versions. To generate the report, run `src/quarto_render.ps1` at the project root. Or similar commands in Linux or MacOS.

1.1 Repository organization

The repository is organized as follows:

- `data`: not committed due to size.
 - `data/stocks_1min_returns` and `data/stocks_5min_returns`: individual stock log-returns at the 1-minute and 5-minute frequencies. Stocks are ID’d by their CRSP PERMNO. The raw data `data/trades.db` is not committed and is from the TAQ database.
 - `data/ff6_1min_returns_raw.csv`, `data/ff6_1min_returns.csv` and `data/ff6_5min_returns.csv`: Factor returns at the 1-minute and 5-minute frequencies. The raw data is available at www.sakethaleti.com/data.
- `src`: Most files here are sections of the report.
 - `src/data_factors.ipynb` `src/data_stocks.ipynb`: Data checking and processing.
 - `src/pca.ipynb`: High-frequency PCA of the factors estimation.
 - `src/betas.ipynb`: Realized betas and risk-adjusted returns estimation.
 - `src/alphas.ipynb`: Conditional alphas estimation.
 - `src/parameters.py`, `src/utls.py`: Utility scripts.
- `bibliography`: reference papers.
- `docs`: report of this empirical analysis.
- Configuration files:
 - Docs and repository: `__quarto.yml` `README.md` `.gitignore`
 - Python: `.python-version` `pyproject.toml` `uv.lock`

2 Data processing: factors

This notebook mines and processes the factor data needed. Read the [README.md](#)/Chapter 1 for context about the project.

2.1 Data and methodology

2.1.1 Raw data

Sources: Fama-French 6 factors from sakethaleti.com/data, or [this Google Drive link](#). Documentation can be found in the section 1 of the [online appendix](#).

Format: Already at the 1-minute level. One column for the minute, and one for each factor. The factors are in returns (not log).

Filters:

- Sample period (same as stocks):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are included.
 - Hours: regular trading hours (09:30 - 16:00 ET).
 - Days: regular trading days, removing days with market-wide halts and diminished trading hours.

Qualities:

- The data should be order by `datetime`.
- The returns should be in decimal format (e.g., 0.01 for 1%).
- There shouldn't be data for non-trading days. Specially: there shouldn't be data for a stock and not for the factors; and there should be all 390 minutes of a day with data.
- The minutes should be left labelled as 9:30 - 9:30:00 -> 9:30:59, 9:31 - 9:31:00 -> 9:31:59, ..., 15:59 - 15:59:00 -> 15:59:59, and no additional 16:00 data point.

All are guaranteed by the source, and checked informally. The last is not, and we might need to transform it.

2.1.2 Methodology

The steps to process the data are as below.

- Download the data into "data/ff6_1min_returns_raw.csv". Alternatively, could be read directly from Google Drive.
- Apply the filters above.
- Check if the minutes are labelled correctly, and transform if needed.
- Transform data into log-returns.
- Aggregate the data into left labeled 5-minute blocks (09:30, 09:35, ..., 15:55), by summing the returns of the label minute and the 4 minutes before it. Save it into "data/ff6_5min_returns.csv".

2.2 Code setup

Codes for debugging:

```
%reset -f
import importlib
```

Core modules:

```
import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import polars as pl
import numpy as np

import pandas_market_calendars as mcal

import plotnine as gg
from plotnine import ggplot, aes

import src.utils as ut
importlib.reload(ut)
from src.parameters import PARAMETERS as PARS
```

2.3 Importing and dates handling

Importing and formatting dates as datetime. First look at the data:

```
data = (
    pl.read_csv("data/ff6_1min_returns_raw.csv")
    .with_columns(
        pl.col("datetime")
        .str.strptime(pl.Datetime, "%Y-%m-%d %H:%M:%S", )
    )
)
cols_fct = ["ff__mkt", "ff__smb", "ff__hml", "ff__rmw", "ff__cma", "ff__umd"]

print(f>Data shape: {data.shape}")
print("First rows:")
data.head(3)
```

Data shape: (2454420, 7)

First rows:

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
1996-01-02 09:30:00	-0.002509	0.001133	-0.000592	-0.000541	-0.000922	-0.003535
1996-01-02 09:31:00	-0.000061	0.000184	0.000053	0.000002	-0.000021	0.000204

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
1996-01-02 09:32:00	-0.000123	0.000121	-0.000045	0.000123	-0.000098	-0.000083

Checking sorting:

```
if not data["datetime"].is_sorted():
    raise ValueError("Data is not sorted by datetime.")
```

All missing dates are non-trading days in NYSE's calendar (assumed as the same as NASDAQ and AMEX).

```
date_days = data["datetime"].dt.date().unique()
date_gaps = date_days.diff(1, "drop").dt.total_days()

date_miss = []
for day, gap in zip(
    date_days[1:].filter(date_gaps > 1),
    date_gaps.filter(date_gaps > 1)
):
    date_miss.extend(
        np.arange(day - np.timedelta64(gap - 1, "D"), day)
        .astype("datetime64[D]")
    )
date_miss = np.array(date_miss, dtype = "datetime64[D]")

date_valid = (
    mcal.get_calendar("NYSE") # Same as NASDAQ and AMEX
    .valid_days(PARS.date_start, PARS.date_end)
    .to_numpy("datetime64[D]")
)

date_miss_valid = date_miss[np.isin(date_miss, np.array(date_valid))]

if date_miss_valid.size > 0:
    raise ValueError(
        f"There are {len(date_miss_valid)} gaps in days. They begin at:" +
        ut.format_days_byyear(date_miss_valid)
    )
```

Checking non-full days (less than 391 minutes of data). All are special days given by the NYSE's calendar. Will be removed.

```
dates_non_full = PARS.trading_days[
    ~ np.isin(PARS.trading_days, PARS.trading_days_factors)
]
print(f"Non-full trading days:" + ut.format_days_byyear(dates_non_full))

data = data.filter(np.isin(data["datetime"].dt.date(), PARS.trading_days_factors))
```

Non-full trading days:

- 2001: 07-03, 11-23, 12-24
- 2002: 07-05, 11-29, 12-24
- 2003: 07-03, 11-28, 12-24, 12-26
- 2004: 11-26
- 2005: 11-25
- 2006: 07-03, 11-24
- 2007: 12-24
- 2015: 11-27, 12-24
- 2016: 11-25
- 2017: 07-03, 11-24

Removing the 16:00 added stamp and the 9:30 overnight return:

```
data = data.filter(  
  pl.col("datetime").dt.time().is_between(  
    pl.time(9, 30), pl.time(16, 0), closed = "none"  
  )  
)
```

2.4 Log-returns and 5-minute aggregation

Data is in returns, and indeed all values are in the range $[-1, 1]$. We can transform it into log-returns by applying the map $r \rightarrow \log(1 + r)$.

```
data_1min = data.with_columns(  
  pl.col(cols_fct).log1p()  
)  
  
data_1min
```

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2001-01-02 09:31:00	0.000095	-0.00038	-0.000202	0.000268	0.000314	-0.000224
2001-01-02 09:32:00	-0.000312	-0.00004	0.00021	0.000598	0.000524	0.000227
2001-01-02 09:33:00	-0.000307	-0.000065	0.000177	0.000557	0.000068	-0.000511
2001-01-02 09:34:00	-0.001305	0.000494	0.000684	0.000274	0.000593	0.000511
2001-01-02 09:35:00	-0.000827	0.000199	0.000638	0.00066	0.000439	0.000085
...	...	...	...	...	...	...
2017-12-29 15:55:00	-0.000386	-0.000364	-0.000003	0.000018	0.000022	-0.000021
2017-12-29 15:56:00	-0.00003	0.00007	-0.000064	0.000003	-0.00009	0.000055
2017-12-29 15:57:00	-0.000033	0.000039	-0.00004	0.000023	-0.000023	0.000103
2017-12-29 15:58:00	-0.000192	-0.000075	-0.000107	0.000056	0.000017	0.000075
2017-12-29 15:59:00	5.8310e-7	-0.000083	-0.000132	-0.000041	0.000018	-0.000061

Aggregating below. It is important that the minute blocks are (9:30, 9:35, ...).

```
data_5min = (
    data_1min.groupby_dynamic("datetime", every="5m")
    .agg([pl.col(col).sum() for col in cols_fct])
)
```

```
data_5min
```

datetime	ff__mkt	ff__smb	ff__hml	ff__rmw	ff__cma	ff__umd
datetime[s]	f64	f64	f64	f64	f64	f64
2001-01-02 09:30:00	-0.001828	0.000009	0.000868	0.001697	0.001499	0.000003
2001-01-02 09:35:00	-0.000719	-0.001081	0.000788	0.002525	0.000066	-0.000575
2001-01-02 09:40:00	0.00256	-0.002862	-0.002178	0.000718	-0.000589	-0.002095
2001-01-02 09:45:00	-0.000581	-0.000438	-0.000018	-0.000494	0.00174	-0.000943
2001-01-02 09:50:00	-0.000909	-0.000451	0.000601	0.000339	0.002158	-0.000354
...	...	...	...	...	...	...
2017-12-29 15:35:00	-0.00055	0.000362	-0.000056	0.000039	-0.000106	0.000181
2017-12-29 15:40:00	-0.000458	-0.000037	-0.00016	0.000045	0.000087	0.000097
2017-12-29 15:45:00	0.000247	-0.000531	0.000275	-0.000324	0.000167	-0.000144
2017-12-29 15:50:00	-0.001065	-0.000379	0.000221	-0.000101	0.000517	-0.000404
2017-12-29 15:55:00	-0.000642	-0.000412	-0.000346	0.000059	-0.000056	0.00015

Finally, saving as .csv:

```
data_1min.write_csv("data/ff6_1min_returns.csv")
data_5min.write_csv("data/ff6_5min_returns.csv")
```

3 Data processing: stocks

This notebook mines and processes the stock data needed. Read the [README.md](#)/Chapter 1 file for context about the project.

3.1 Data and methodology

3.1.1 Raw data

Sources: Intraday tick data from the [NYSE TAQ database](#). The relevant documentation is [Daily_TAQ_Client_Spec_v4.3.pdf](#).

Format:

The data is can be easily stored as an SQLite3 database with the main table **trades** of the following schema:

```
CREATE TABLE trades (  
  permno    INTEGER NOT NULL, -- Permanent number of the stock  
  ts_ny     INTEGER NOT NULL, -- Timestamp in New York time (epoch seconds)  
  price     REAL     NOT NULL, -- Price of the trade  
  size      INTEGER NOT NULL, -- Size of the trade  
  ex        TEXT     NOT NULL, -- Exchange code  
  corr      TEXT     NOT NULL, -- Correction code  
  ticker    TEXT     NOT NULL, -- Stock ticker symbol  
  ambiguous INTEGER NOT NULL, -- Flag if the trade is ambiguous (0 or 1)  
  PRIMARY KEY (permno, ts_ny)  
) WITHOUT ROWID
```

Further documentation might be found looking for the relevant websites.

For the confection of this notebook, the data was given in a private Dropbox folder. The TODO file contains a script to download it into the `data/stocks_raw/trades.db` file.

Qualities:

The data is expected to have the qualities below. All are guaranteed by the data source.

- Within each **permno**, the data should be ordered by **ts_ny**. *Will be sorted.*
- There is one obs per minute, at the 00 seconds mark. Duplicates don't exist but missing minutes do. *Will be checked.*
- Data validity: **price** should never be negative or zero; **ex** and **corr** should have only the correct levels. *Anomalies are filtered out.*
- There should be data only for trading days. *Will be checked.*

3.1.2 Methodology

First, some initial filters are applied (Section 3.3) and the raw data is saved in an optimized format. Then, the data is transformed from a permno ~ second ~ price data into a permno ~ minute ~ log return data (Section 3.5).

The calculations are done via DuckDB SQL queries.

3.2 Code setup

Codes for debugging and running SQL inside notebooks:

```
%reset -f
import importlib

from IPython.core.magic import register_cell_magic
from pprint import pprint

import duckdb

queries_registered: dict[str, str] = {}

@register_cell_magic
def register_query(line: str, cell: str) -> None:
    query_name = line.strip()
    if not query_name:
        raise ValueError("You must provide a name for the query.")
    queries_registered[query_name] = cell
    return None
```

Setting up DuckDB infrastructure. The threads and memory limits are set following the memory constraints (16GB RAM) of the machine used to process the data.

```
%%register_query duckdb_infra
INSTALL sqlite;
LOAD sqlite;

-- DETACH db_trades;
ATTACH IF NOT EXISTS 'data/stocks_raw/trades.db' AS db_trades (TYPE SQLITE);

SET enable_progress_bar = true;
SET enable_progress_bar_print = true;

PRAGMA threads = 7;
PRAGMA memory_limit = '14GB';
```

```
duckdb.execute(queries_registered["duckdb_infra"])
```

Core modules:

```

import os
if os.path.basename(os.getcwd()) == "src": os.chdir("..")

import math

import polars as pl
import numpy as np

import src.utils as ut
importlib.reload(ut)
from src.parameters import PARAMETERS as PARS

```

3.3 Initial data cleaning

An initial filter is applied before the processing logic, to remove unnecessary data.

The column types are converted to ones that occupy less space in memory. See the [DuckDB types](#) for reference.

The is then saved into a [.parquet](#) file, a columnar storage format that is more efficient for analytical queries and compression than the original SQLite3 database. The row group size is chosen as approximately the median of observations per permno.

Filters:

- Exchanges: NYSE, NYSE-Arca, NASDAQ, and AMEX (code shares 10 and 11 at the CRSP database) (`ex IN ('P', 'N', 'Q', 'A')`).
- Types of trades: regular trades (`corr = '00'`).
- Price should be positive (`price > 0`).
- Sample period(same as factors):
 - Range: 2001-01-02 to 2017-12-29. Stocks that are listed or delisted during this period are kept.
 - Hours: regular trading hours (09:30:00 - 15:59:59 ET).

Formatting and data checking:

- `permno`: is a 5-digit positive integer, and is stored as UINT32.
- `ts_ny`:
 - Is a seconds-since-1970-01-01 timestamp.
 - Our start date is 2001-01-02, which guarantees that the relevant observations are always positive.
 - There are no duplicates (`ts_ny > LAG(ts_ny)`).
 - There is only one observation per minute at the 00 mark `ts_ny % 60 != 0`, thus it is transformed into a minutes epoch.
 - Stored as UINT32, whose maximum date goes up to the year 10141.
- `price`: 14 integer digits and 6 decimal digits, stored as FLOAT8. `Decimal(9, 6)` could be used, but python's matrix libraries work better with floats.

The two checks are stored into a validation column, and the final result is partitioned by it. Rows are partitioned into the `data/stocks_raw` directory's sub-folders `valid=pass/`, `valid=fail_not_minute/`, and `valid=fail_unordered_or_dupe/` (not committed to git).

This will be implemented via the following query:

```

%%register_query raw_to_clean

-- ?1: start epoch seconds (PARS.date_start)
-- ?2: end epoch seconds (PARS.date_end)

COPY (
    SELECT
        CAST(permno AS UINT32) AS permno,
        CAST(ts_ny // 60 AS UINT32) AS ts_min_ny,
        CAST(price AS FLOAT8) AS price,
        CASE
            WHEN (COUNT(*) OVER (PARTITION BY permno, ts_min_ny)) > 1
                AS is_duplicate
            THEN 'fail_duplicate'
            ELSE 'pass'
        END AS valid
    FROM db_trades.trades
    WHERE
        ts_ny BETWEEN ?1 AND ?2
        AND corr = '00'
        AND ex IN ('P', 'N', 'Q', 'A')
        AND (ts_ny % 86400) BETWEEN 34200 AND 57599 -- 09:30:00 to 15:59:59
        AND price > 0
    ORDER BY permno, ts_min_ny -- Costly but helps with querying later
) TO 'data/stocks_raw' (
    FORMAT PARQUET,
    COMPRESSION 'ZSTD',
    ROW_GROUP_SIZE 555000, -- Median of observations per permno
    PARTITION_BY (valid)
);

```

Then, we check if there are no data in the valid==fail_duplicate/ folders, and move the valid=pass/ data into the data/stocks_raw folder:

```

if False: # Only execute once
    duckdb.execute(
        queries_registered["raw_to_clean"],
        [
            int(x.astype("datetime64[s]").astype(np.int64))
            for x in (PARS.date_start, PARS.date_end)
        ]
    )
    os.rename(
        "data/stocks_raw/valid=pass/data_0.parquet",
        "data/stocks_raw/trades_clean.parquet"
    )

if "valid==fail_duplicate/" in os.listdir("data/stocks_raw/"):
    raise ValueError(
        "There are invalid rows to be analyzed. Check the `data/stocks_raw/`'s"
        " subfolders"
    )

```

For utility, save the number of observations for each permno:

```
%%register_query counts_permno
COPY (
  SELECT
    permno,
    COUNT(*) as obs_count
  FROM 'data/stocks_raw/trades_clean.parquet'
  GROUP BY permno
  ORDER BY permno
) TO 'data/stocks_raw/counts_permno.csv' (HEADER, DELIMITER ',');
```

```
if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno"])

data_counts_permno = pl.read_csv("data/stocks_raw/counts_permno.csv")
```

The number of observations (1-minute blocks) for each (permno, day):

```
%%register_query counts_permno_day_1min
COPY (
  SELECT
    permno,
    (ts_min_ny // (60 * 24)) as d,
    COUNT(*) as obs_count
  FROM 'data/stocks_raw/trades_clean.parquet'
  GROUP BY permno, d
) TO 'data/stocks_raw/counts_permno_day_1min.csv' (HEADER, DELIMITER ',');
```

```
if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno_day_1min"])
```

And the number of 5-minute blocks with data, for each (permno, day):

```
%%register_query counts_permno_day_5min
COPY (
  SELECT
    permno,
    (ts_min_ny // (60 * 24)) as d,
    COUNT(DISTINCT ts_min_ny // 5) as obs_count
  FROM 'data/stocks_raw/trades_clean.parquet'
  GROUP BY permno, d
) TO 'data/stocks_raw/counts_permno_day_5min.csv' (HEADER, DELIMITER ',');
```

```
if False: # Only execute once
    duckdb.execute(query = queries_registered["counts_permno_day_5min"])
```

Saving the data, and filling missing days within the permno lifetime with obs_count = 0:

```
data_counts_min = {
    x: pl.read_csv(f"data/stocks_raw/counts_permno_day_{x}min.csv")
    for x in [1, 5]
}
```

3.4 Dates analysis

Now, we turn to analyzing the distribution of observations within a day for a stock. We want to remove stocks that have too many days with none or too few observations.

3.4.1 Trading days

First, we need to define what is a missing day, that is, what are the trading days. In [src/parameters.py](#), we calculate the full (9:30 → 16:00) trading days as the union of: days with full factor data, days as described by the `exchange_calendars` python package, and days described by the `pandas_market_calendars` package. But, there might be some days that were not full trading days for the stocks in our sample.

First, let's check if any of the stocks have data for non-trading days. It is expected that there are data for non-full trading days.

```
data_days = data_counts_min[1].with_columns(
    pl.from_epoch(pl.col("d"), time_unit = "d")
)

days_unique = np.unique(data_days["d"].to_numpy().astype("datetime64[D]"))

if not np.isin(days_unique, PARS.trading_days).all():
    raise ValueError("There are days in the data that are not trading days.")

if not np.isin(days_unique, PARS.trading_days_factors).all():
    print("There are days in the data that are not full trading days.")
```

There are days in the data that are not full trading days.

The river plot below shows, for each day, the quantiles of the number of minutes with data for each stock, divided by the total number of minutes in a trading day (390).

The blue dots show days where the factor data didn't have all minutes with data, which at first, is our truth source for non-full days. The gray dots show the days where the median is -0.1 points smaller than its 5-day rolling mean, that is, possible extraordinary trading days.

```
data_days_fill = (
    data_days
    .with_columns(pl.col("obs_count") / PARS.blocks_1min)
    .group_by("d")
    .agg(
        median = pl.median("obs_count"),
        **{
            f"q{q}": pl.quantile("obs_count", q / 10)
        }
    )
```

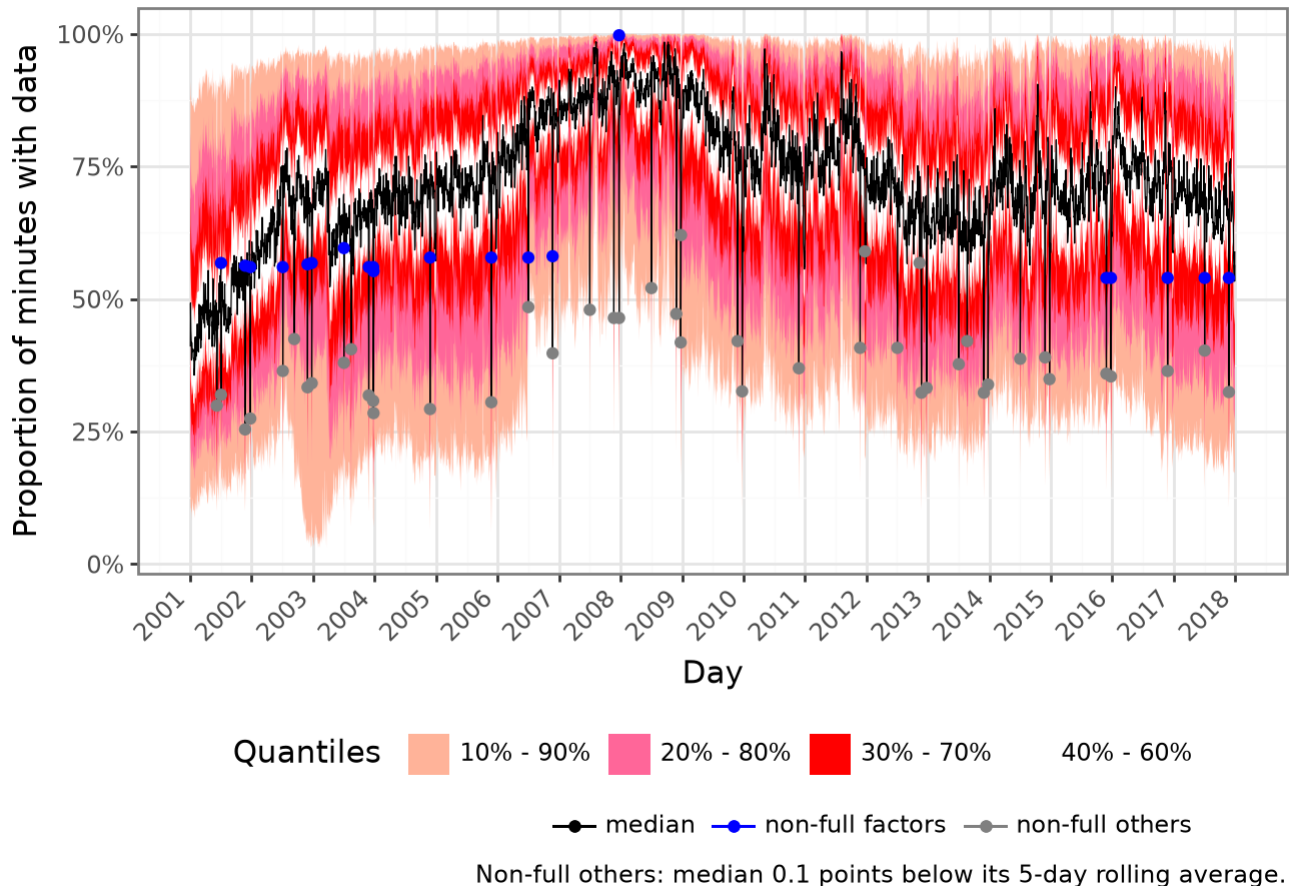
```

        for q in [4, 6, 3, 7, 2, 8, 1, 9]
    }
)
.sort("d")
)

ut.plot_day_fullness_river(data_days_fill, data_counts_permno.shape[0])

```

Proportion of minutes with data within each day
Quantiles calculated each day across all 2364 stocks



The average median is ≈ 0.72 . We can see many outlier dates with very low coverage, which are likely to be extraordinary trading days. Still, some of these days are not within the list of full trading days, as shown below. Individually checking the dates, they still seem to be extraordinary trading days, but that have data present for the factors. They will be included in the non-full trading days list.

```

days_low_obs = (
    data_days_fill
    .with_columns(median_rw = pl.col("median").rolling_mean(8))
    .filter(pl.col("median") - pl.col("median_rw") <= - 0.1)
    ["d"].to_numpy()
)

print(f"Number of days with low observations: {len(days_low_obs)}")
print(ut.format_days_byyear(days_low_obs[
    np.isin(days_low_obs, PARS.trading_days_factors)
]))

```

Number of days with low observations: 52

- 2001: 06-08
- 2002: 09-11
- 2003: 08-15
- 2005: 12-23
- 2007: 07-03, 11-23
- 2008: 07-03, 11-28, 12-24, 12-26
- 2009: 11-27, 12-24
- 2010: 11-26, 12-27
- 2011: 08-15, 11-25, 12-23, 12-27
- 2012: 07-03, 11-12, 11-23, 12-24
- 2013: 07-03, 08-22, 11-29, 12-24
- 2014: 07-03, 11-28, 12-24, 12-26
- 2015: 10-12
- 2016: 12-23

3.4.2 Days with missing data

Now, on to checking which days have missing data. Missing days within the PERMNO's lifetime are filled with 0. For each 1-minute and 5-minute block, we plot:

1. The histogram of the number of observed blocks for each day, across all days of all stocks.
2. Boxplots of the proportion of days with more observed blocks than some threshold τ , across all stocks, for multiple thresholds.

```
data_filled_min = {  
    x: ut.fill_missing_days(data_counts_min[x])  
    for x in [1, 5]  
}
```

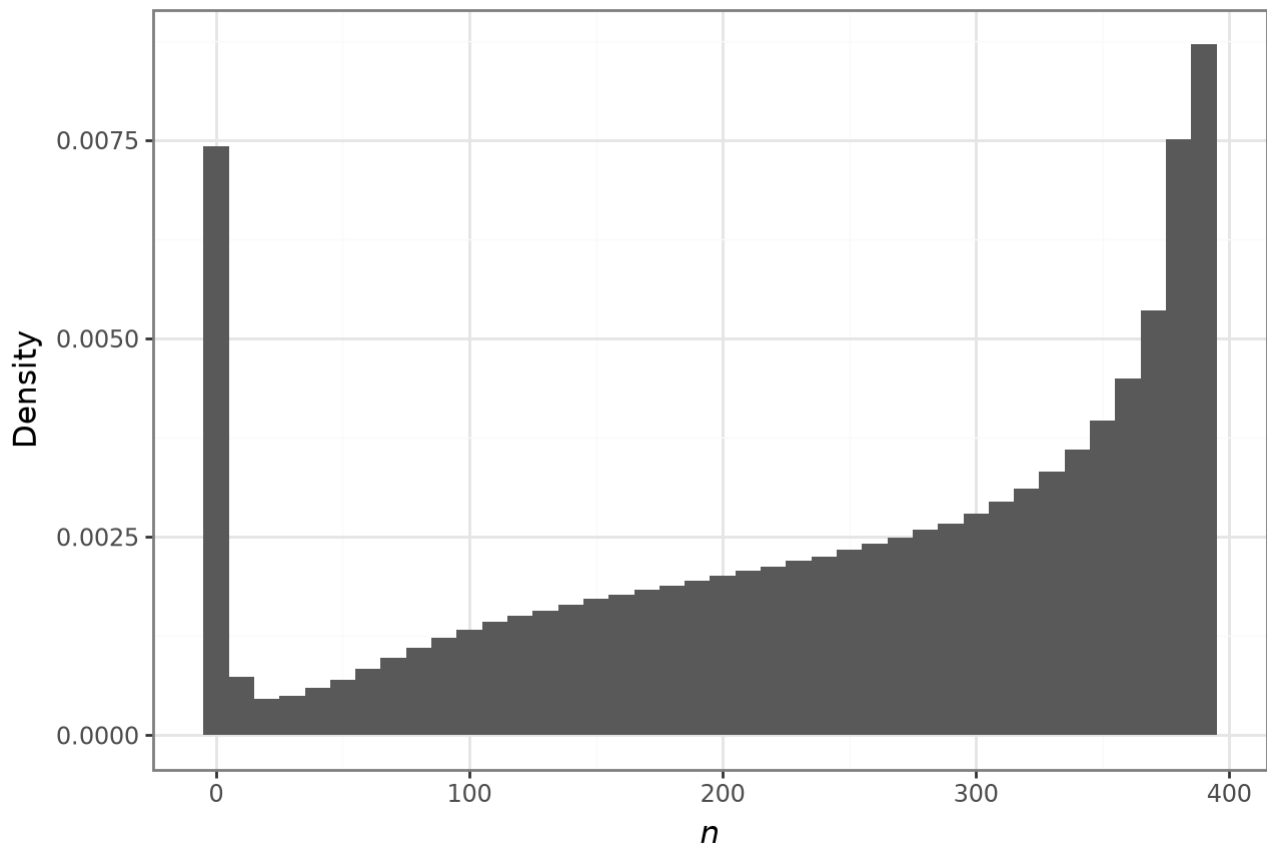
3.4.3 1-Minute blocks

For $\tau = 78$, there are 1,292 stocks with at most 2.5% non-full days.

```
ut.plot_day_fullness_hist(data_filled_min[1])
```

Distribution of days' observation count (n)

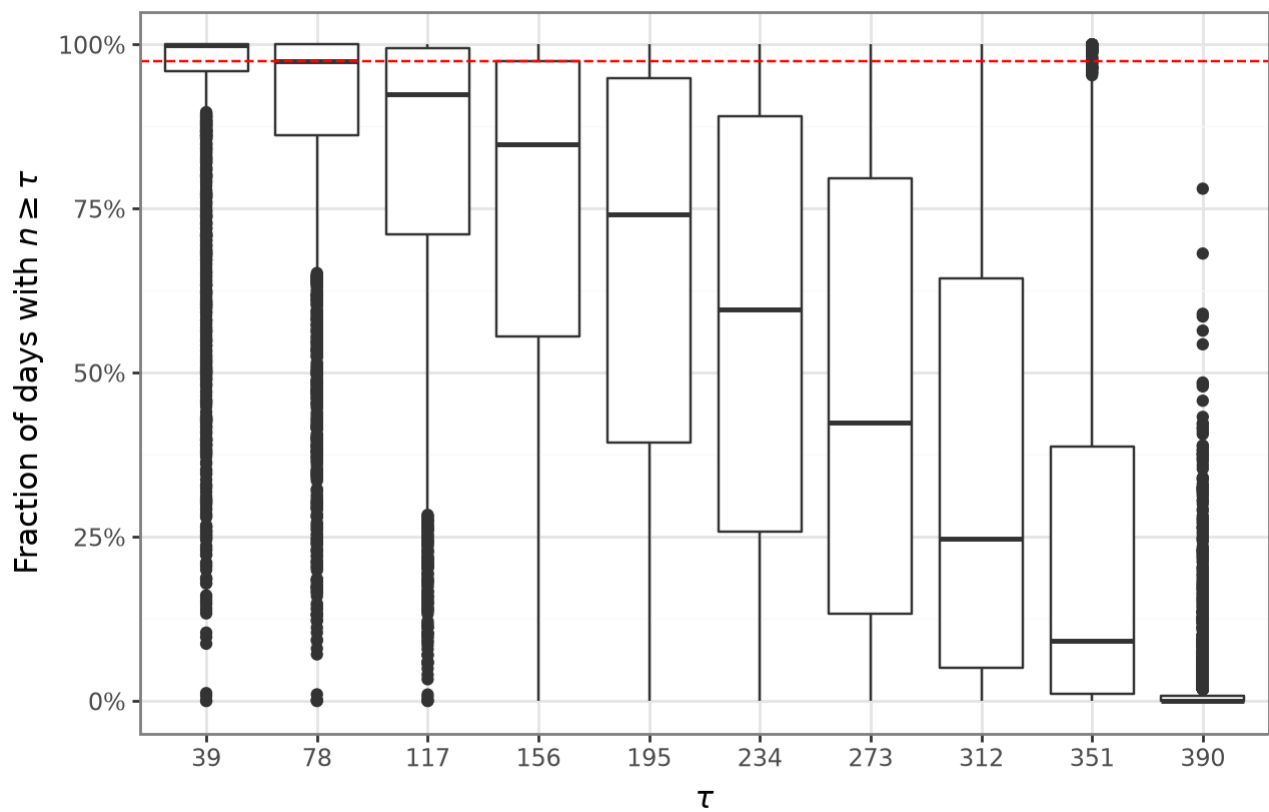
Across all 6,397,249 stocks-day pairs



```
ut.plot_day_fullness_box(data_filled_min[1], 1, data_counts_permno.shape[0])
```

Distribution of days' observation count (n)

Across all 2364 stocks



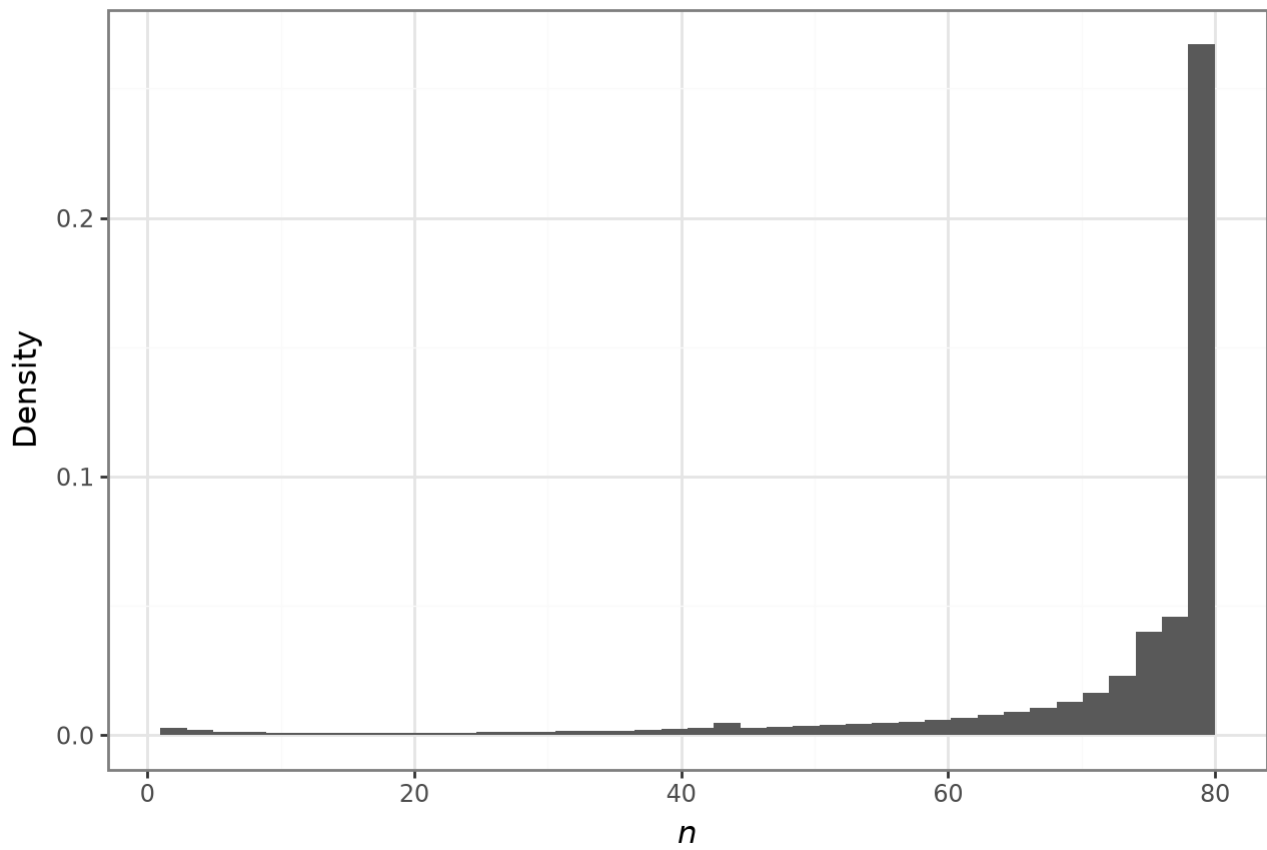
Red line: 97.5%.

3.4.4 5-minute blocks

```
ut.plot_day_fullness_hist(data_counts_min[5])
```

Distribution of days' observation count (n)

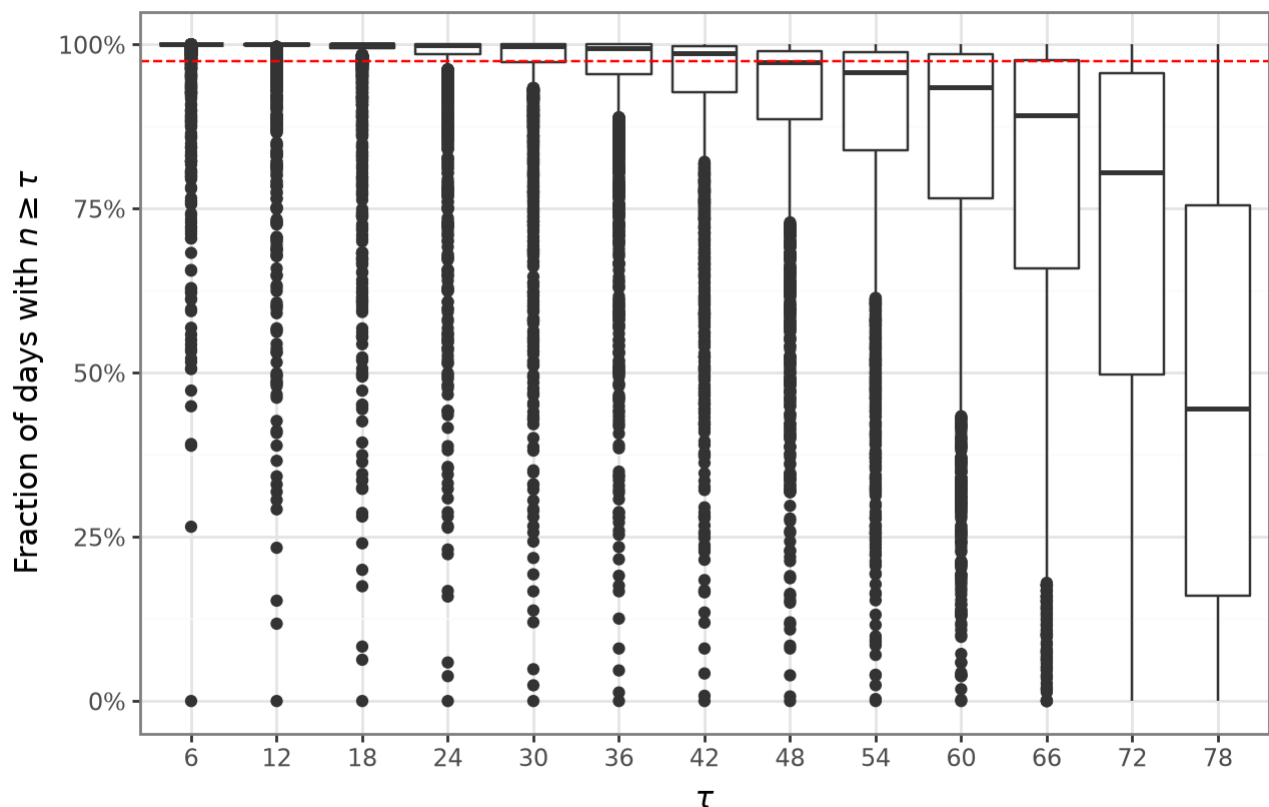
Across all 6,054,309 stocks-day pairs



```
ut.plot_day_fullness_box(data_counts_min[5], 5, data_counts_permno.shape[0])
```

Distribution of days' observation count (n)

Across all 2364 stocks



Red line: 97.5%.

3.5 Aggregation of 1-minute log returns

To aggregate the data, for each `permno`:

1. For each 1-minute block m : get the last seen price before the end of m 's block, p_m .
2. Minimal liquidity filter: at most 2.5% of the trading days (excluding pre-listing and post-delisting days) have less than 78 minutes with data.
3. Calculate $r_m^1 = \ln(p_t/p_{m-1})$.
4. Exclude the first return of the day to remove overnight effects.
5. Exclude non-full trading days..

- Save a [data/stocks_1min_returns.parquet](#) file with the columns `permno`, `ts_min_ny`, and `log_return`.

Some notes:

- As checked in the raw data, the observations are already at the minute level with no duplicates, so we don't need to actually do any aggregation.
- The "last seen price" is not the undisputed correct price to use for calculating returns. Other methods include the midpoint of the bid-ask spread and the volume-weighted average price.
- We could save the information of how many observations were within each minute block, but that information is not needed at the moment.

The transformation could be done with either DuckDB or Polars, both multithreaded. It also could be done in chunks if the data is too big. Polars seems to deal better with the latter, but the following DuckDB code is written to process the data in one go. See the appendix for alternatives.

```
%%register_query trade_returns_aggregation_chunked

-- ?1: list of permnos to filter
-- ?2: chunk file path

COPY (
  WITH
  trades_by_min AS (
    SELECT
      permno,
      ts_min_ny,
      price,
      (ts_min_ny // (60 * 24)) AS d
    FROM 'data/stocks_raw/trades_clean.parquet'
    WHERE
      permno BETWEEN ?1[0] AND ?1[-1]
  )
  SELECT
    permno,
    ts_min_ny,
    LN(price / LAG(price, 1) OVER w) AS log_return
  FROM trades_by_min
  WINDOW w AS (PARTITION BY permno, d ORDER BY ts_min_ny)
  QUALIFY
    LAG(price, 1) OVER w IS NOT NULL
)
TO ?2 (
  FORMAT PARQUET,
  COMPRESSION 'ZSTD',
  ROW_GROUP_SIZE 555000
);
```

```
duckdb.execute(queries_registered["trade_returns_aggregation_chunked"])
```

The data will be forward filled with 0 return on minutes with no trades, and will also be aggregated to the 5-minute level. But, storing these to disk is unneeded, and it is better to create the variations on-the-fly.

3.5.1 Visualizing raw data and aggregation

Lets visualize the work to have a first look at the data and check if everything is as expected.

First, lets sample some random stocks.

```
vis_permnos = data_counts_permno["permno"].sample(4).to_numpy()
```

We could visualize the original raw data, but very few operations were done on it. Still, one could run the following SQL and polars codes to generate a dataset to visualize:

```

%%register_query raw_permno_sample

-- ?1: list of integer permnos to filter

SELECT
    CAST(permno AS UINT32) AS permno,
    CAST(ts_ny // 60 AS UINT32) AS ts_min_ny,
    CAST(price AS FLOAT8) AS price,
FROM db_trades.trades
WHERE
    permno IN ?1

```

```

data_permno_raw = (
    duckdb.sql(queries_registered["raw_permno_sample"], params = [vis_permnos.tolist()])
    .pl()
    .with_columns(
        datetime = pl.from_epoch(pl.col("ts_min_ny") * 60, time_unit = "s")
    )
    .select(["permno", "datetime", "price"])
)

```

The code below plots the clean prices and the returns aggregation. Both are aggregated to the 30-day level, to make the relationship between the two clearer.

The relationship between the two looks as expected.

```

ut.plot_prices_returns(
    pl.scan_parquet("data/stocks_raw/trades_clean.parquet")
    .filter(pl.col("permno").is_in(vis_permnos))
    .collect(),
    pl.scan_parquet("data/stocks_1min_returns.parquet")
    .filter(pl.col("permno").is_in(vis_permnos))
    .collect()
)

```

Price and log returns across time

Each column is a randomly selected PERMNOs



3.6 Appendix

If the calculations above are too harsh for the available machine's memory, one can do it in chunks of `permno` values. The chunks are defined below, striving for a balance in the number of rows in each chunk. Each chunk has 7% of the rows, a number defined by the memory constraints (16GB RAM) of the machine used to process the data.

```
agg_chunk_fraction = 0.07
agg_chunk_cutoff = math.floor(
    int(data_counts_permno["obs_count"].sum()) * agg_chunk_fraction * 0.95
) # 0.95 avoids last chunk being too small
agg_total_chunks = math.ceil(1 / agg_chunk_fraction)

chunks = []
current_stock = 0

for i in range(agg_total_chunks):
    c = []
    c_rows = 0
    while c_rows < agg_chunk_cutoff and current_stock < len(data_counts_permno):
        c.append(data_counts_permno["permno"][current_stock])
        c_rows += data_counts_permno["obs_count"][current_stock]
        current_stock += 1
    if i == agg_total_chunks - 1:
```

```

        c.extend(data_counts_permno["permno"][current_stock:].to_list())
        chunks.append(c)

if sum(len(c) for c in chunks) != len(data_counts_permno):
    raise ValueError("Some stocks are missing from the chunks.")

print(f"Number of chunks: {len(chunks)}")
print("Number of stocks in each chunk: ", end = "")
pprint([len(c) for c in chunks], width = 50, compact = True)
print("Rows/1M per chunk (approx): ", end = "")
pprint([
    round(
        sum(data_counts_permno.filter(pl.col("permno").is_in(c))["obs_count"])
        / 1_000_000,
        1
    )
    for c in chunks
], width = 50, compact = True)

```

```

Number of chunks: 15
Number of stocks in each chunk: [168, 328, 103, 106, 111, 110, 106, 141, 142, 150,
166, 164, 163, 190, 216]
Rows/1M per chunk (approx): [107.0, 108.1, 107.7, 107.6, 107.1, 107.5, 107.0,
106.9, 107.3, 107.7, 107.9, 106.9, 107.1, 107.0,
103.3]

```

```

data_trades_lazy = pl.scan_parquet(
    "data/stocks_raw/trades_clean.parquet",
    parallel = "row_groups",
    low_memory = False
)

def lazy_pipeline_agg(chunk: list[int]) -> pl.LazyFrame:
    trading_days_extra = PARS.trading_days[
        ~ np.isin(PARS.trading_days, PARS.trading_days_final)
    ].astype("int32")

    pipeline = (
        data_trades_lazy
        .filter(pl.col("permno").is_between(chunk[0], chunk[-1]))
        #.sort(["permno", "ts_min_ny"]) # Already sorted
        .with_columns(
            d = (pl.col("ts_min_ny") // (60 * 24))
        )
        .with_columns(
            log_return = (
                pl.col("price") / (
                    pl.col("price")
                    .shift(1).over(["permno", "d"]) # Never cross days
                )
                # order_by = "ts_min_ny" is unneeded
            )
        )
    )

```

```

        ).log(),
    )
    .filter(
        pl.col("log_return").is_not_null() # Excludes first null return of day
        & (~ pl.col("ts_min_ny").is_in(trading_days_extra))
    )
    .select(["permno", "ts_min_ny", "log_return"])
)
return pipeline

```

For each chunk, the pipeline is executed and the results are saved in a parquet file such as [data/stocks_1min_returns/chunk_1.parquet](#):

```

if False: # Only execute once
    for i, chunk in enumerate(chunks[-1:]):
        print(f"Processing chunk {i+1}/{len(chunks)} with {len(chunk)} permnos")
        lazy_pipeline_agg(chunk).sink_parquet(
            f"data/stocks_1min_returns/chunk_{i}.parquet",
            compression = "zstd",
            engine = "streaming",
            row_group_size = 555_000
        )

```

An alternative to the code for Polars is the DuckDB code below.

```

%%register_query trade_returns_aggregation_chunked

-- ?1: list of permnos to filter
-- ?2: chunk file path

COPY (
    WITH
    trades_by_min AS (
        SELECT
            permno,
            ts_min_ny,
            price,
            (ts_min_ny // (60 * 24)) AS d
        FROM 'data/stocks_raw/trades_clean.parquet'
        WHERE
            permno BETWEEN ?1[0] AND ?1[-1]
    )
    SELECT
        permno,
        ts_min_ny,
        LN(price / LAG(price, 1) OVER w) AS log_return
    FROM trades_by_min
    WINDOW w AS (PARTITION BY permno, d ORDER BY ts_min_ny)
    QUALIFY
        LAG(price, 1) OVER w IS NOT NULL
)

```

```
TO ?2 (  
    FORMAT PARQUET,  
    COMPRESSION 'ZSTD',  
    ROW_GROUP_SIZE 555000  
);
```

```
if False: # Run polars instead  
    for i, chunk in enumerate(chunks[-1:]):  
        print(f"Processing chunk {i+1}/{len(chunks)} with {len(chunk)} permnos")  
        duckdb.execute(  
            queries_registered["trade_returns_aggregation_chunked"],  
            [chunk, f"data/stocks_1min_returns/chunk_{i}.parquet"]  
        )
```

4 Factor PCA

To be created...

5 Realized betas and risk-adjusted returns

To be created...

6 Conditional alphas

To be created...